Edge cases (search comes back empty) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

The exact pieces:

- A conditional edge **after** `search_node`: `has_results(state)` returns `"no_grounding"` or `"reason"`.
- A `grounded` field in state, so a caller can check the answer's status without reading its text.
- `make_search_node(search_collection)` — a function that builds the node for whichever collection you pass (Doc07's factory idea), so one graph can search the coffee docs or the empty store.

Then the common case: an off-topic question still gets the *closest* chunks back, so the list isn't empty. Doc08's `score` is a **distance** — the best match is the **smallest** score, and anything above a `MAX_DISTANCE` you tuned isn't real grounding. A middle band (close, but not close enough to cover everything) deserves its own honest answer: "partial".

Hint 2 — The plan, and almost the whole thing

```
best_distance(sources): the smallest source["score"]
has_relevant_results:   empty or best > MAX_DISTANCE -> "no_grounding"
grounding_level:        empty / > MAX_DISTANCE  -> "no_grounding"
                        > GOOD_DISTANCE         -> "partial"
                        otherwise               -> "reason"
partial_grounding_node: "answer only what the context supports, and say
                        what it doesn't cover"
```

One piece to get you unstuck:

```
def best_distance(sources: list[dict]) -> float:
    best = sources[0]["score"]
    for source in sources:
        if source["score"] < best:
            best = source["score"]
    return best
```

Full solution: Show me the solution